

Assignment 4: Large Scale Machine Learning

Sotirios Oikonomou

Task 1: Distributed Machine Learning

1.1 – Hyperparameter Tuning

Question 1 – Problem Setup and Default Parameters

The Forest Cover Type dataset was loaded directly via the scikit-learn API (`fetch_covtype`) and subsampled to 20,000 instances to accommodate the memory constraints of the `ssc.small` VM flavor. The dataset contains 54 features and 7 target classes representing different forest cover types. A `RandomForestClassifier` was instantiated with default parameters, which were inspected using the `get_params()` method. The key defaults relevant to this task are `n_estimators=100`, `max_depth=None` (trees grow until leaves are pure), and `ccp_alpha=0.0` (no pruning). Three-fold cross-validation was applied consistently across both the baseline and the tuning pipeline to produce reliable accuracy estimates. The baseline model achieved a cross-validation accuracy of **0.7025**, establishing the reference point for evaluating the tuning results.

Three VMs of `ssc.small` flavor were provisioned on the Swedish Science Cloud running Ubuntu 22.04, as shown in Figure 1. Ray was installed on all three nodes, with `ray-head-sotiris` acting as the cluster coordinator and the two worker nodes joining via the head's private IP.

☐	ray-worker-2-sotiris	Ubuntu 22.04 - 2024.01.15	192.168.2.122	ssc.small	de2-course-key2	Active		nova	None	Running	1 minute	Create Snapshot	▼
☐	ray-worker-1-sotiris	Ubuntu 22.04 - 2024.01.15	192.168.2.241	ssc.small	de2-course-key2	Active		nova	None	Running	3 minutes	Create Snapshot	▼
☐	ray-head-sotiris	Ubuntu 22.04 - 2024.01.15	192.168.2.206, 130.238.27.58	ssc.small	de2-course-key2	Active		nova	None	Running	4 minutes	Create Snapshot	▼

Question 2 – Distributed Hyperparameter Tuning Results

A distributed hyperparameter tuning pipeline was implemented using Ray Tune with an exhaustive Grid Search strategy over the following parameter grid:

- `max_depth`: [5, 10, 20]
- `n_estimators`: [50, 100, 200]
- `ccp_alpha`: [0.0, 0.001, 0.01]

This produced 27 unique combinations, each evaluated via 3-fold cross-validation. The pipeline was deployed and run three times — using 1, 2, and 3 VMs respectively — to measure the effect of horizontal scaling on wall-clock time. The head node connected to the existing Ray cluster using `ray.init(address='auto')`, allowing Ray Tune to automatically distribute trials across all available CPUs.

The best hyperparameter combination found was `max_depth=20`, `n_estimators=200`, `ccp_alpha=0.0`, achieving a cross-validation accuracy of **0.7013**. This is marginally lower than the default accuracy of 0.7025, which can be explained by the fact that the default `max_depth=None` allows trees to grow deeper than the maximum value of 20 tested in the grid. The result confirms that more estimators and deeper trees are preferable for this dataset, while pruning via `ccp_alpha` consistently degraded performance.

The timing results across the three configurations are summarised in the table below:

VMs	CPUs	Tuning Time (s)	Speedup
1	1.0	186.87	1.0×
2	2.0	123.36	1.5×
3	3.0	90.00	2.1×

The speedup is sublinear relative to the number of VMs, which is expected due to coordination overhead between the Ray head and worker nodes, as well as the fixed cost of loading data and initialising the runtime environment on each trial. Nevertheless, the results clearly demonstrate the advantage of distributing the workload — a 3-VM cluster completed the same search in less than half the time of a single node.

```

secon — ubuntu@ray-head-sotiris: ~ — ssh -A -i ~/.ssh/de2-course-key ubuntu@130.238.27.58 — 148x39
| train_rfc_7492e_00003 | TERMINATED | 10 | 50 | 0 | 0.648604 | 1 | 2.14283 | 0.0549497 |
| train_rfc_7492e_00004 | TERMINATED | 10 | 50 | 0.001 | 0.614405 | 1 | 2.17151 | 0.0747676 |
| train_rfc_7492e_00005 | TERMINATED | 10 | 50 | 0.01 | 0.572404 | 1 | 2.16099 | 0.0638036 |
| train_rfc_7492e_00006 | TERMINATED | 20 | 50 | 0 | 0.694852 | 1 | 3.07016 | 0.0493294 |
| train_rfc_7492e_00007 | TERMINATED | 20 | 50 | 0.001 | 0.624805 | 1 | 4.02093 | 0.0720541 |
| train_rfc_7492e_00008 | TERMINATED | 20 | 50 | 0.01 | 0.558455 | 1 | 4.0421 | 0.0759626 |
| train_rfc_7492e_00009 | TERMINATED | 5 | 100 | 0 | 0.574954 | 1 | 2.34975 | 0.055104 |
| train_rfc_7492e_00010 | TERMINATED | 5 | 100 | 0.001 | 0.575054 | 1 | 2.39749 | 0.0590285 |
| train_rfc_7492e_00011 | TERMINATED | 5 | 100 | 0.01 | 0.530453 | 1 | 2.37655 | 0.0602278 |
| train_rfc_7492e_00012 | TERMINATED | 10 | 100 | 0 | 0.651404 | 1 | 3.8013 | 0.0551824 |
| train_rfc_7492e_00013 | TERMINATED | 10 | 100 | 0.001 | 0.618155 | 1 | 3.87853 | 0.0744134 |
| train_rfc_7492e_00014 | TERMINATED | 10 | 100 | 0.01 | 0.565155 | 1 | 3.85653 | 0.0687682 |
| train_rfc_7492e_00015 | TERMINATED | 20 | 100 | 0 | 0.697102 | 1 | 5.80807 | 0.0465233 |
| train_rfc_7492e_00016 | TERMINATED | 20 | 100 | 0.001 | 0.628655 | 1 | 7.70214 | 0.0718097 |
| train_rfc_7492e_00017 | TERMINATED | 20 | 100 | 0.01 | 0.559255 | 1 | 7.66362 | 0.073857 |
| train_rfc_7492e_00018 | TERMINATED | 5 | 200 | 0 | 0.573654 | 1 | 4.23376 | 0.0552392 |
| train_rfc_7492e_00019 | TERMINATED | 5 | 200 | 0.001 | 0.573154 | 1 | 4.24587 | 0.0596719 |
| train_rfc_7492e_00020 | TERMINATED | 5 | 200 | 0.01 | 0.531753 | 1 | 4.23597 | 0.0626262 |
| train_rfc_7492e_00021 | TERMINATED | 10 | 200 | 0 | 0.652154 | 1 | 7.07711 | 0.0562567 |
| train_rfc_7492e_00022 | TERMINATED | 10 | 200 | 0.001 | 0.614255 | 1 | 7.22824 | 0.077955 |
| train_rfc_7492e_00023 | TERMINATED | 10 | 200 | 0.01 | 0.571155 | 1 | 7.13063 | 0.0699547 |
| train_rfc_7492e_00024 | TERMINATED | 20 | 200 | 0 | 0.701302 | 1 | 10.8838 | 0.0501521 |
| train_rfc_7492e_00025 | TERMINATED | 20 | 200 | 0.001 | 0.632505 | 1 | 14.675 | 0.0692262 |
| train_rfc_7492e_00026 | TERMINATED | 20 | 200 | 0.01 | 0.557455 | 1 | 14.6602 | 0.0737525 |

=====
RESULTS
=====
Best max_depth : 20
Best n_estimators : 200
Best ccp_alpha : 0.0
Best CV accuracy : 0.7013
Default accuracy : 0.7025
Total tuning time : 186.87 seconds
CPUs in cluster : 1.0
=====
ubuntu@ray-head-sotiris:~$

```

secon — ubuntu@ray-head-sotiris: ~ — ssh -A -i ~/.ssh/de2-course-key ubuntu@130.238.27.58 — 148x39

train_rfc_974d4_00003	TERMINATED	10	50	0	0.648604	1	3.20249	0.0549497
train_rfc_974d4_00004	TERMINATED	10	50	0.001	0.614405	1	2.25187	0.0747676
train_rfc_974d4_00005	TERMINATED	10	50	0.01	0.572404	1	2.22087	0.0638036
train_rfc_974d4_00006	TERMINATED	20	50	0	0.694852	1	4.98536	0.0493294
train_rfc_974d4_00007	TERMINATED	20	50	0.001	0.624805	1	4.1271	0.0720541
train_rfc_974d4_00008	TERMINATED	20	50	0.01	0.558455	1	4.12018	0.0759626
train_rfc_974d4_00009	TERMINATED	5	100	0	0.574954	1	4.33043	0.055104
train_rfc_974d4_00010	TERMINATED	5	100	0.001	0.575054	1	2.39433	0.0590285
train_rfc_974d4_00011	TERMINATED	5	100	0.01	0.530453	1	3.54562	0.0602278
train_rfc_974d4_00012	TERMINATED	10	100	0	0.651404	1	3.81963	0.0551824
train_rfc_974d4_00013	TERMINATED	10	100	0.001	0.618155	1	3.89475	0.0744134
train_rfc_974d4_00014	TERMINATED	10	100	0.01	0.565155	1	5.57661	0.0687682
train_rfc_974d4_00015	TERMINATED	20	100	0	0.697102	1	5.71311	0.0465233
train_rfc_974d4_00016	TERMINATED	20	100	0.001	0.628655	1	10.4916	0.0718097
train_rfc_974d4_00017	TERMINATED	20	100	0.01	0.559255	1	7.63175	0.073857
train_rfc_974d4_00018	TERMINATED	5	200	0	0.573654	1	4.35346	0.0552392
train_rfc_974d4_00019	TERMINATED	5	200	0.001	0.573154	1	6.40612	0.0596719
train_rfc_974d4_00020	TERMINATED	5	200	0.01	0.531753	1	4.25936	0.0626262
train_rfc_974d4_00021	TERMINATED	10	200	0	0.652154	1	10.0464	0.0562567
train_rfc_974d4_00022	TERMINATED	10	200	0.001	0.614255	1	7.27432	0.077955
train_rfc_974d4_00023	TERMINATED	10	200	0.01	0.571155	1	7.32275	0.0699547
train_rfc_974d4_00024	TERMINATED	20	200	0	0.701302	1	15.283	0.0501521
train_rfc_974d4_00025	TERMINATED	20	200	0.001	0.632505	1	15.0239	0.0692262
train_rfc_974d4_00026	TERMINATED	20	200	0.01	0.557455	1	20.6628	0.0737525

RESULTS

Best max_depth : 20
Best n_estimators : 200
Best ccp_alpha : 0.0
Best CV accuracy : 0.7013
Default accuracy : 0.7025
Total tuning time : 123.36 seconds
CPUs in cluster : 2.0

ubuntu@ray-head-sotiris:~\$

secon — ubuntu@ray-head-sotiris: ~ — ssh -A -i ~/.ssh/de2-course-key ubuntu@130.238.27.58 — 148x39

train_rfc_6f6b2_00003	TERMINATED	10	50	0	0.648604	1	2.18378	0.0549497
train_rfc_6f6b2_00004	TERMINATED	10	50	0.001	0.614405	1	3.22753	0.0747676
train_rfc_6f6b2_00005	TERMINATED	10	50	0.01	0.572404	1	3.19859	0.0638036
train_rfc_6f6b2_00006	TERMINATED	20	50	0	0.694852	1	3.1717	0.0493294
train_rfc_6f6b2_00007	TERMINATED	20	50	0.001	0.624805	1	5.63074	0.0720541
train_rfc_6f6b2_00008	TERMINATED	20	50	0.01	0.558455	1	5.56215	0.0759626
train_rfc_6f6b2_00009	TERMINATED	5	100	0	0.574954	1	2.37986	0.055104
train_rfc_6f6b2_00010	TERMINATED	5	100	0.001	0.575054	1	3.642	0.0590285
train_rfc_6f6b2_00011	TERMINATED	5	100	0.01	0.530453	1	2.47018	0.0602278
train_rfc_6f6b2_00012	TERMINATED	10	100	0	0.651404	1	5.42012	0.0551824
train_rfc_6f6b2_00013	TERMINATED	10	100	0.001	0.618155	1	3.91717	0.0744134
train_rfc_6f6b2_00014	TERMINATED	10	100	0.01	0.565155	1	5.60623	0.0687682
train_rfc_6f6b2_00015	TERMINATED	20	100	0	0.697102	1	8.18694	0.0465233
train_rfc_6f6b2_00016	TERMINATED	20	100	0.001	0.628655	1	7.73386	0.0718097
train_rfc_6f6b2_00017	TERMINATED	20	100	0.01	0.559255	1	10.6728	0.073857
train_rfc_6f6b2_00018	TERMINATED	5	200	0	0.573654	1	4.45703	0.0552392
train_rfc_6f6b2_00019	TERMINATED	5	200	0.001	0.573154	1	6.382	0.0596719
train_rfc_6f6b2_00020	TERMINATED	5	200	0.01	0.531753	1	6.35745	0.0626262
train_rfc_6f6b2_00021	TERMINATED	10	200	0	0.652154	1	7.16446	0.0562567
train_rfc_6f6b2_00022	TERMINATED	10	200	0.001	0.614255	1	10.2078	0.077955
train_rfc_6f6b2_00023	TERMINATED	10	200	0.01	0.571155	1	10.3516	0.0699547
train_rfc_6f6b2_00024	TERMINATED	20	200	0	0.701302	1	11.0163	0.0501521
train_rfc_6f6b2_00025	TERMINATED	20	200	0.001	0.632505	1	20.0825	0.0692262
train_rfc_6f6b2_00026	TERMINATED	20	200	0.01	0.557455	1	20.3623	0.0737525

RESULTS

Best max_depth : 20
Best n_estimators : 200
Best ccp_alpha : 0.0
Best CV accuracy : 0.7013
Default accuracy : 0.7025
Total tuning time : 90.00 seconds
CPUs in cluster : 3.0

ubuntu@ray-head-sotiris:~\$

1.2 – Distributed Training of Deep Neural Networks

When scaling deep learning to large datasets or large models, two fundamentally different parallelism strategies emerge: data parallelism and model parallelism. Understanding when to apply each is essential for designing efficient distributed training pipelines.

In data parallelism, the model is replicated across multiple devices and each replica processes a different batch of training data simultaneously. After each forward pass, gradients are aggregated — typically via an AllReduce operation — and all replicas update their weights identically. This is the dominant approach in practice and is directly supported by both PyTorch's `DistributedDataParallel` and TensorFlow's `MirroredStrategy`. It is most appropriate when the model fits comfortably in the memory of a single GPU but the dataset is large enough that training on a single device would be prohibitively slow. The main bottleneck shifts from computation to synchronization, which becomes significant as the number of nodes grows.

Model parallelism takes a different approach: the model itself is partitioned across devices, with each device responsible for a subset of layers or parameters. Data flows sequentially through the partitions during the forward pass, and gradients propagate back through the same path. This strategy is necessary when the model is too large to fit into a single GPU's memory — a constraint increasingly common with large language models and transformer architectures with billions of parameters.

Regarding hyperparameter tuning, distributing trials is the right choice when individual training runs are computationally manageable — as demonstrated in Task 1.1, where each Random Forest trial completed in seconds. In this regime, task-level parallelism through frameworks like Ray Tune delivers near-linear speedup with minimal overhead. However, when training a single model already requires multiple GPUs due to memory or time constraints — for instance, fine-tuning a large transformer on a massive corpus — distributing the training itself becomes the bottleneck to address, and applying data or model parallelism within each trial is necessary before considering trial-level distribution. In practice, both levels can be combined: Ray Tune dispatching trials where each trial internally runs a distributed training job.

Task 2: Federated Learning

Simulation Setup

The federated learning simulation was implemented using PySyft 0.8.7 in Google Colab. Two in-process Datasite servers were created using `sy.Worker(name=..., dev_mode=True)`, with root clients obtained via `server.root_client`. The MNIST training data was uploaded to each server as a private `sy.Dataset` containing two `sy.Asset` objects (features `X` and labels `y`). Asset pointers — references backed by `server_uid` and `action_id` identifiers — were returned to the orchestrator in place of the raw tensors, enforcing data locality from the outset. Remote training functions were defined using the `@sy.syft_function` decorator with `ExactMatch` input policies and `SingleExecutionExactOutput` output policies, ensuring each function could only access the specific private assets it was authorised for and could only be executed once.

Two virtual workers, Client A and Client B, were instantiated as in-process Datasite servers. The full MNIST training set (60,000 samples) was partitioned into two non-overlapping subsets of 30,000 samples each. Client A received samples 0–29,999 and Client B received samples 30,000–59,999, ensuring no data overlap between clients.

Training

Each client's training function was submitted via `client.code.request_code_execution()` and executed remotely through the `@sy.syft_function` API. The global model — a three-layer fully connected network (784→128→64→10) — was initialised with a fixed random seed (`torch.manual_seed(42)`) inside each function to ensure both clients started from identical weights, as required for correct FedAvg aggregation. Each client trained locally for 2 epochs using SGD (`lr=0.01`, `momentum=0.9`). Only the resulting `model.state_dict()` was returned to the orchestrator — raw training data never left the respective Datasite server. The server aggregated the two updates using FedAvg, computing an element-wise average of the corresponding weight tensors.

Analysis

In PySyft 0.7, when a tensor was sent to a `VirtualWorker`, the data scientist's local environment received a `PointerTensor` in its place — a reference object that points to the tensor on the remote worker. Any operation called on the `PointerTensor` was serialised and executed remotely, with only the result returned. Crucially, calling `.get()` on raw data required explicit permission from the worker, and in a real deployment this would be controlled by the data owner. This design means the data scientist can instruct computation on private data without ever seeing the actual values. In PySyft 0.8+, the same concept is enforced through `ActionObject`, which wraps tensors uploaded to a Datasite and governs all operations via configurable input and output policies (`ExactMatch`, `SingleExecutionExactOutput`). In the simulation, this is enforced directly through the PySyft 0.8.7 API: the `@sy.syft_function` decorator with `ExactMatch` and `SingleExecutionExactOutput` policies ensures the training function executes entirely within the Datasite server's scope, and only the `state_dict` is returned to the orchestrator — no raw features or labels are ever exposed.

Model	Test Accuracy
Federated (Client A + Client B)	0.9588
Single worker (Client A only)	0.9591
Improvement	-0.03%

Federated model accuracy : 0.9588 (95.88%)
Single worker (Client A) : 0.9591 (95.91%)
Improvement : -0.03%

The federated model achieved 95.88% test accuracy compared to 95.91% for the single-worker baseline — a negligible gap of 0.03 percentage points. This result is expected for IID-partitioned MNIST: each client receives a representative subset of all 10 digit classes, so the aggregated model gains no meaningful distributional advantage over a well-trained single-client model. The key takeaway is that FL incurs essentially no accuracy cost on IID data while maintaining strict privacy — raw MNIST samples never left their respective Datasite server throughout training.

Written Reflection

Federated learning and traditional distributed ML share the goal of parallelising computation across multiple nodes, but they differ fundamentally in their assumptions. In distributed ML, data is centralised on a cluster, shuffled uniformly across workers, and computation happens on homogeneous machines with high-bandwidth interconnects. The result is that each worker sees a statistically representative slice of the overall dataset.

In federated learning, neither assumption holds. Statistical heterogeneity arises because each client's data reflects its own local distribution — a hospital's patient population, a phone user's typing habits — which may differ substantially from other clients. This non-IID (non-independent and identically distributed) data makes naive FedAvg less stable: the global model must reconcile conflicting gradient directions, which can slow convergence or bias the model toward clients with more data or more training rounds. System heterogeneity compounds the problem: clients are diverse devices — mobile phones, IoT sensors, hospital servers — with vastly different compute speeds, memory, and network reliability. Stragglers can

delay round completion, and dropped clients mid-round require the system to decide whether to wait or proceed with partial updates.

The Parameter Server (PS) pattern is architecturally similar to FL: a central server holds the global model, workers pull the latest parameters, compute gradients locally, and push updates back. The server aggregates and applies them. FL effectively implements this pattern but under adversarial real-world conditions. The key differences are motivation and trust: in a PS setup, all workers are owned by the same organisation, data is centralised, and communication is reliable. In FL, workers are independent parties with private data and no obligation to be available or honest. This introduces additional concerns — such as poisoning attacks, where a malicious client submits corrupted updates — that do not exist in a classical PS environment.